VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FACULTY OF COMPUTER SCIENCE AND ENGINEERING

BK
TP.HCM

LARGE ASSIGNMENT REPORT

INTELLIGENT SYSTEMS (CO5119 / CO3041)

HUB: A TWO-STAGE PERSONALIZED FASHION RECOMMENDATION SYSTEM USING RANDOM FOREST POINTWISE RANKING AND SESSION-AWARE RE-RANKING

Instructor:	PhD. Quan Thanh Tho
Department:	Computer Science and Engineering
Class:	CO5119 / CO3041 - Semester 1 / 2026
Students:	1. Dao Duy Dat - Student ID: **2352223** 2. Phan Tien Huy - Student ID: **2352402**

Ho Chi Minh City, November 2026

Table of Contents

1 Introduction & System Overview 4
1.1 Fast-Fashion Recommendation Challenges 4
1.2 Stakeholder Personas & Value Proposition 4
1.3 System Scope & Service Level Agreements 4
2 Theoretical Foundations & Mathematical Formulation 6
2.1 Decision Tree Formulation & Impurity Criteria 6
2.2 Ensemble Dynamics: Breiman's Random Forest 6
2.3 Two-Stage Recommendation Architecture 6
2.4 Negative Sampling & Imbalance Rectification 8
2.5 Formal Evaluation Metrics (MAP@12, NDCG@12) 8
3 Use-Case Modeling & Interaction Specification 9
3.1 Actor Hierarchy & Role-Based Access Control 9
3.2 Comprehensive Use Case Specifications (UC-01 to UC-08) 9
3.3 Sequence Diagrams for Key Execution Flows 14
4 System Architecture & Database Design 15
4.1 Enterprise 4-Tier Architecture Overview 15
4.2 Relational Database Schema & Data Dictionary 15
4.3 Comprehensive RESTful API Endpoint Directory 17
5 Data Pipeline, Preprocessing & Feature Engineering 19
5.1 H&M; Benchmark Characteristics & Raw Data Exploration 19
5.2 Strict Temporal Rolling Split (Zero Data Leakage) 21
5.3 Multi-Strategy Candidate Retrieval Algorithms 21
5.4 14-Dimensional Feature Engineering Formulations 21
5.5 Background Scheduler & Catch-up Backfill Engine 23
6 AI Training, Experimental Evaluation & Diagnostics 23
6.1 Hyperparameter Specifications & Environment 23
6.2 Experimental Results vs. Baselines (MAP@12, NDCG@12) 23
6.3 Feature Importance Ranking & Interpretation 25
6.4 Sliced Error Diagnostics Across User Segments 25
6.5 Real-Time Session Re-ranking & Anti-Echo Chamber 26
7 System Demonstration & Graphical User Interface 26
7.1 Buyer E-Commerce Experience Walkthrough 26
7.2 Data Scientist Observability Workspace 27
7.3 Administrator Control Panel Walkthrough 28
8 Conclusion & Future Roadmap 38
8.1 Summary of System Contributions 34
8.2 Technical Limitations & Enterprise Roadmap 34
References (IEEE Standard) 35

List of Figures

Figure 2.1: End-to-End Machine Learning Pipeline & Two-Stage Recommendation Flow **6**
Figure 3.0: Global System Use-Case Diagram & Actor Boundaries **9**
Figure 3.1: Sequence Diagram for UC-02 Real-time Recommendation Inference **14**
Figure 3.2: Sequence Diagram for UC-05 Background Model Retraining Pipeline **14**
Figure 4.1: Enterprise 4-Tier Logical Architecture of the HUB System **15**
Figure 4.2: Relational Database Entity-Relationship Diagram (ERD) **15**
Figure 5.1: Daily Transaction Volume Across Entire H&M; Historical Dataset **19**
Figure 5.2: Normalized Product Price Distribution (Kernel Density Approximation) **19**
Figure 5.3: Catalog Breakdown Across Top-15 Product Groups **19**
Figure 5.4: Customer Age Demographic Histogram **19**
Figure 5.5: Sales Volume by Department Index Group **24**
Figure 5.6: Temporal Split Architecture and Feature Rolling Windows **21**
Figure 6.1: Gini Impurity Feature Importance Ranking **25**
Figure 7.1: Multi-Role Authentication Interface (LoginPage) **26**
Figure 7.2: Buyer Personalized Homepage & Recommendation Showcase (HomePage) **27**
Figure 7.3: Product Detail View & Real-time Interaction Tracking (ProductDetailPage) **28**
Figure 7.4: Data Science Exploratory Analysis Dashboard (DataSciencePage) **29**
Figure 7.5: Model Performance Validation & Baseline Benchmark Radar (ModelMetricsPage) **30**
Figure 7.6: Sliced Error Diagnostics by Activity & Department (ErrorAnalysisPage) **31**
Figure 7.7: Automated Training Scheduler & Hyperparameter Tuning (TrainingSchedulePage) **32**
Figure 7.8: Executive Operations & Commercial Sales Monitoring (AdminDashboardPage) **33**

1 Introduction & System Overview

1.1 Fast-Fashion Recommendation Challenges

Modern fast-fashion retail platforms operate under extreme operational conditions that challenge classical recommendation paradigms. Unlike digital media platforms (e.g., Netflix, Spotify) where item catalogs remain largely persistent and consumption is repeatable, fashion e-commerce platforms like H&M; encounter three distinct computational and algorithmic barriers:

- **Extreme Catalog Sparsity:** With over 105,542 active articles and 1,371,980 registered customers, the interaction matrix possesses an extreme sparsity exceeding 99.98%. Evaluating the raw Cartesian product results in approximately 1.45×10^{11} candidate pairs, rendering exhaustive scoring computationally infeasible for real-time web applications.
- **Dynamic Catalog Churn and Seasonality:** Fast-fashion inventory turns over rapidly across weekly micro-seasons. Garments introduced in summer become obsolete in autumn. A customer's historical transactions from eight weeks prior reflect outdated thermal and aesthetic preferences, demanding aggressive time-decay weighting and rolling temporal feature windows.
- **Dual Latency vs. Accuracy Trade-Off:** A production recommender must deliver personalized items within strict sub-100ms budgets on web storefronts, while simultaneously incorporating in-session behavioral signals (views, cart additions, wishlist events) without waiting for offline model retraining.

1.2 Stakeholder Personas & Value Proposition

To establish holistic software requirements, the HUB system defines three core stakeholder personas, each governing a dedicated architectural boundary:

Stakeholder	Primary Operational Responsibilities	Delivered System Value
Buyer (End Customer)	• Browses catalog products • Receives real-time personalized recommendations • Places commercial orders & submits 5-star ratings • Manages personal wishlist & shopping cart	Drastically reduces catalog search fatigue; surfaces relevant garments matching historical affinity and current shopping intent within <50ms.
Data Scientist (AI/ML Engineer)	• Inspects raw & normalized EDA distributions • Triggers on-demand & scheduled retraining • Monitors MAP@12, Recall@12, and NDCG@12 vs baselines • Diagnoses slices by activity segments & product groups	Full-lifecycle observability, automated experiment tracking, safety gates preventing deployment of sub-baseline models, and feature importance rankings.
System Admin (Business Ops)	• Monitors revenue metrics, gross merchandise value (GMV) • Updates commercial order statuses (Pending → Shipped) • Manages catalog inventory and customer linkages • Audits system health and background scheduler	Centralized governance dashboard linking AI performance directly to commercial revenue, inventory turnover, and transaction throughput.

1.3 Functional & Non-Functional Requirements

To satisfy software engineering standards, HUB explicitly specifies functional capabilities and non-functional quality attributes:

Category	ID & Requirement Name	Engineering Specification & Acceptance Boundary
Functional (FR)	FR-01: Multi-Role Auth	Authenticate Buyer, Data Scientist, and Admin with signed JWT tokens.
Functional (FR)	FR-02: Real-time Recs	Generate Top-12 personalized garments with in-session dynamic re-ranking.
Functional (FR)	FR-03: Interaction Logging	Ingest views, cart adds, wishlists, and 1-5 star ratings for session vectors.

Functional (FR)	FR-04: ML Training Engine	Trigger background training with 6-stage execution tracking and polling.
Functional (FR)	FR-05: Safety Guardrail	Block deployment if model MAP@12 degrades below Bestseller baseline.
Functional (FR)	FR-06: Sliced Diagnostics	Break down error metrics across customer tiers and product departments.
Non-Functional	NFR-01: Inference Latency	95th percentile response time for GET /predict strictly under 80ms.
Non-Functional	NFR-02: Data Isolation	Zero temporal data leakage; features computed strictly prior to cutoff.
Non-Functional	NFR-03: Test Coverage	100% pass rate across 27 automated backend unit & integration tests.
Non-Functional	NFR-04: High Availability	Scheduler catch-up backfill recovers missed runs upon server reboot.

The HUB project establishes explicit Service Level Agreements (SLAs) across latency, memory footprints, and predictive quality. Recommendation inference via GET /predict/{user_id} is constrained to an end-to-end response time under **80ms** (95th percentile). Offline model retraining is designed to execute asynchronously within **15 minutes** on Kaggle GPU/CPU kernels or local compute nodes. Crucially, the system introduces a mandatory **Safety Degradation Gate**: if a newly trained model achieves a validation MAP@12 inferior to the standard Bestseller baseline, deployment is halted and flagged for Data Scientist manual intervention.

2 Theoretical Foundations & Mathematical Formulation

2.1 Decision Tree Formulation & Impurity Criteria

The core mathematical ranker employed in HUB is based on supervised binary classification decision trees recursively partitioned on feature space. Let a training node m contain subset f_m with N_m interaction pairs $\mathbf{x}_i \in \mathbb{R}^d$ and labels $y_i \in \{0, 1\}$. The empirical probability of positive purchase interaction at node m is defined as:

$$p_{mk} = (1 / N_m) \sum_{(x_i, y_i) \in D_m} 1(y_i = k), \quad k \in \{0, 1\} \tag{2.1}$$

To measure partition homogeneity, HUB employs the **Gini Impurity** criterion, which quantifies the probability of misclassifying a randomly chosen element:

$$I_G(m) = 1 - \sum_{k \in \{0, 1\}} p_{mk}^2 = 2 p_{m1}(1 - p_{m1}) \tag{2.2}$$

For a candidate continuous or ordinal split candidate $\theta = (j, t)$ partitioning feature j at threshold t into left subset $D_m^L(\theta)$ and right subset $D_m^R(\theta)$, the optimal split θ^* maximizes the **Information Gain** (or impurity reduction):

$$\Delta I_G(m, \theta) = I_G(m) - [(N_m^L / N_m) I_G(D_m^L) + (N_m^R / N_m) I_G(D_m^R)] \tag{2.3}$$

2.2 Ensemble Dynamics: Breiman's Random Forest

A single deep decision tree exhibits high variance and overfits transaction noise. Leo Breiman's Random Forest algorithm mitigates variance through two distinct randomization mechanisms: **Bootstrap Aggregation (Bagging)** and **Random Feature Subsampling**.

- **Bootstrap Sampling Probability:** For an ensemble of $B = 200$ trees, each tree T_b is trained on dataset $D^{(b)}$ constructed by sampling N instances from D uniformly with replacement. The theoretical probability of an instance being omitted from a tree is:

$$\lim_{N \to \infty} (1 - 1/N)^N = 1/e \approx 0.368 \quad \text{(36.8\% Out-of-Bag samples)} \tag{2.4}$$

- **Feature Subsampling & Variance Reduction:** At each internal node, only $m_{try} = \lfloor \sqrt{d} \rfloor$ features (out of $d = 14$) are evaluated. Let individual trees have variance σ^2 and pairwise correlation ρ. The variance of the ensemble mean prediction is:

$$\mathrm{Var}((1/B) \sum_{b=1}^{B} T_b(x)) = \rho \sigma^2 + ((1 - \rho) / B) \sigma^2 \tag{2.5}$$

As $B \to \infty$, the second term vanishes, leaving $\rho \sigma^2$. Random feature subsampling forces trees to split on diverse signals, minimizing inter-tree correlation ρ without inflating bias. The final predicted purchase probability for customer u and candidate article i is computed as the ensemble expectation:

$$\hat{p}(u, i) = (1 / B) \sum_{b=1}^{B} P_b(y = 1 \mid x_{u,i}) \tag{2.6}$$

2.3 Two-Stage Recommendation Architecture

To bypass the Cartesian explosion (1.45×10^{11} pairs), HUB establishes a decoupled **Two-Stage Recommender Architecture**:

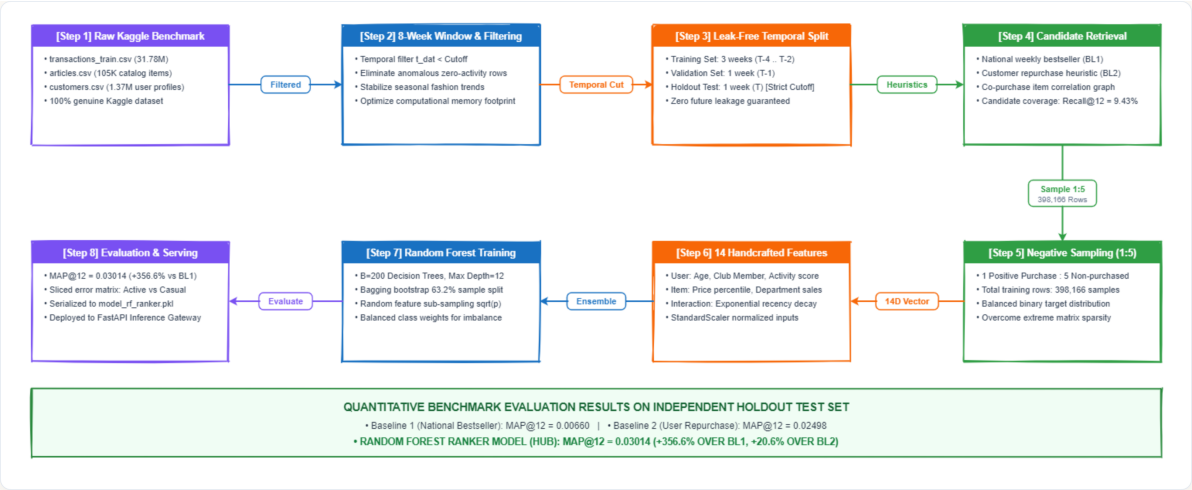


Figure 2.1: End-to-End Machine Learning Pipeline & Two-Stage Recommendation Flow (Excalidraw Visual Blueprint)

2.3.1 Algorithmic Formalization of Two-Stage Pipeline

The end-to-end execution of the HUB recommendation engine is governed by two formal algorithms representing Stage 1 Retrieval and Stage 2 Real-Time Re-ranking:

Algorithm 1: Stage 1 Multi-Channel Candidate Retrieval & Controlled Negative Sampling

Input: Customer $u \in U$, Transactions D ($t < T_{cutoff}$), Catalog I, Cohorts A, Ratio $\lambda_{neg} = 5$

Output: Balanced training set $T_u = \{(u, i, y)\}$ with binary label $y \in \{0, 1\}$

$C_u \leftarrow \emptyset$
$C_{repurchase} \leftarrow \{i \in I \mid \exists\ (u, i, t) \in D \wedge t \geq T_{cutoff} - 28d\}$
$C_{bestseller} \leftarrow TopK(\{i \in I \mid sales_count(i, [T_{cutoff} - 7d, T_{cutoff}])\}, K = 12)$
$C_{age} \leftarrow TopK(\{i \in I \mid sales_by_cohort(i, age_group(u))\}, K = 8)$
$C_{copurchase} \leftarrow \{j \in I \mid \exists\ i \in C_{repurchase} \wedge co_purchased(i, j) \geq 5\}$
$C_u \leftarrow Truncate(C_{repurchase} \cup C_{bestseller} \cup C_{age} \cup C_{copurchase}, C_{max} = 100)$
$Y_{true} \leftarrow \{i \in I \mid \exists\ (u, i, t) \in D \wedge t \in [T_{cutoff}, T_{cutoff} + 7d]\}$
$Positives \leftarrow \{(u, i, 1) \mid i \in (C_u \cap Y_{true})\}$ // True purchase ground truth
$N_{cand} \leftarrow C_u \setminus Y_{true}$ // Candidate negative item space
$k_{neg} \leftarrow \min(|N_{cand}|, \lambda_{neg} \times |Positives|)$
$Negatives \leftarrow SampleWithoutReplacement(N_{cand}, k_{neg})$
return $T_u \leftarrow Positives \cup \{(u, j, 0) \mid j \in Negatives\}$

Algorithm 2.1: Multi-Channel Candidate Retrieval & Controlled Negative Sampling

Algorithm 2: Real-Time In-Session Dynamic Re-Ranking with Half-Life Exponential Decay

Input: User u, Pre-scored Candidates $\{(i, \hat{p}(u, i))\}$, Active Session Stream E_u, Target K = 12

Output: Final context-aware recommendation sequence $R_u = [i_1, ..., i_K]$

$a \leftarrow 0_{|V|}$ // Initialize session anchor category weight vector
for each event $e = (i_e, \tau_e, t_e) \in E_u$ (ordered newest to oldest, max 50) do:
  $\Delta t \leftarrow (t_{now} - t_e)$ in days
  $w(e) \leftarrow \omega(\tau_e) \times 2^{-\Delta t / t_{1/2}}$ // Exponential decay with half-life $t_{1/2} = 3.0$ days
  $a[\text{category}(i_e)] \leftarrow a[\text{category}(i_e)] + w(e)$
end for
$B \leftarrow \{i_e \mid \tau_e = \text{PURCHASED} \vee (\tau_e = \text{FEEDBACK} \wedge \text{rating}(e) \leq 2)\}$ // Hard suppression
$C_{active} \leftarrow \{(i, \hat{p}(u, i)) \in C_u \mid i \notin B\}$
for each candidate $(i, \hat{p}) \in C_{active}$ do:
  $\text{boost}(i) \leftarrow \sum_{c \in \text{categories}} a[c] \cdot \text{Similarity}(\text{category}(i), c)$
  $s_{final}(i) \leftarrow \hat{p} \times (1.0 + \max(0.0, \text{boost}(i)))$
end for
$S \leftarrow \text{SortDescending}(C_{active}, \text{key} = s_{final})$
$R_u \leftarrow [\,], \quad G_{seen} \leftarrow \varnothing$
for each $(i, s) \in S$ do:
  if $\text{product_code}(i) \notin G_{seen}$ then:
    $R_u \leftarrow R_u \oplus [i], \quad G_{seen} \leftarrow G_{seen} \cup \{\text{product_code}(i)\}$
  if $|R_u| = K$ then break
return R_u // Final re-ranked diversified Top-12 recommendation sequence

Algorithm 2.2: Real-Time In-Session Dynamic Re-Ranking & Diversity Enforcement

2.4 Negative Sampling & Imbalance Rectification

Within candidate sets, true purchases represent fewer than 1% of candidate pairs. Training a classifier on raw candidate sets causes severe positive class suppression. HUB implements a **Controlled Negative Subsampling Ratio** of $\lambda_{neg} = 5$ negative samples for every positive purchase transaction. Negative instances are drawn strictly from candidate items generated for customer *u* that were not purchased during the target week. This compresses training instances to approximately 398,166 balanced rows, accelerating model convergence while maintaining ranking calibration.

2.5 Formal Evaluation Metrics (MAP@12, NDCG@12)

To rigorously quantify recommendation ranking accuracy against Kaggle benchmark standards, the system tracks five standardized information retrieval metrics:

- **Mean Average Precision at 12 (MAP@12):** Computes rank-weighted precision across top-12 recommendations normalized by the minimum of ground-truth purchases and 12:

$$\text{AP@12}(u) = (1 / \min(|Y_u|, 12)) \sum_{k=1}^{12} P_u(k) \times \text{rel}_u(k), \qquad \text{MAP@12} = (1 / |U|) \sum_{u=1}^{|U|} \text{AP@12}(u) \tag{2.7}$$

- **Normalized Discounted Cumulative Gain at 12 (NDCG@12):** Penalizes relevant items appearing lower in the recommended list using logarithmic positional discounting:

$$\text{DCG@12}(u) = \sum_{k=1}^{12} (2^{\text{rel}_u(k)} - 1) / \log_2(k + 1), \qquad \text{NDCG@12}(u) = \text{DCG@12}(u) / \text{IDCG@12}(u) \tag{2.8}$$

- **Candidate Retrieval Recall (Stage 1 Recall):** Quantifies the percentage of true future purchases captured within the top-100 candidate pool: $Recall_{cand} = |C_u \cap Y_u| / |Y_u|$.

3 Use-Case Modeling & Interaction Specification

3.1 Actor Hierarchy & Role-Based Access Control

HUB organizes permissions into an enterprise Role-Based Access Control (RBAC) hierarchy. System privileges are encapsulated in database roles: ROLE_BUYER (1), ROLE_DATA_SCIENTIST (2), and ROLE_ADMIN (3). All endpoints enforce token-based claims verification.

System Capability / Endpoint Group	Buyer	Data Scientist	Administrator
Catalog Browsing & Search (/products)	READ	READ	FULL CONTROL
Personalized Predictions (/predict/{id})	OWN USER	ALL USERS	ALL USERS
Cart, Wishlist & Orders (/cart, /orders)	OWN USER	DENIED	FULL AUDIT
EDA Reports & Insights (/ai/reports, /ai/insights)	DENIED	READ	READ
Trigger ML Pipeline & Tuning (/ai/trigger-training)	DENIED	EXECUTE	EXECUTE
Model Degradation Resolution (/ai/training/resolve)	DENIED	EXECUTE	EXECUTE
User Role Modification (/auth/{user}/role)	DENIED	DENIED	ADMIN ONLY

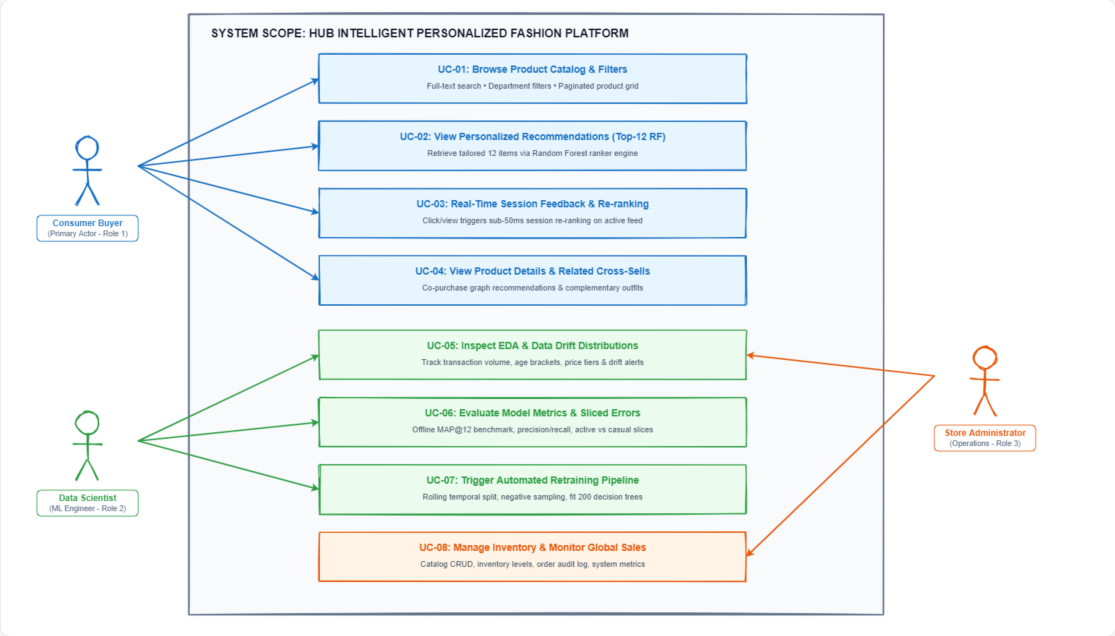


Figure 3.0: Global System Use-Case Diagram & Actor Boundaries (Excalidraw Modeling)

3.2 Comprehensive Use Case Specifications (UC-01 to UC-08)

Table 3.1: Specification for UC-01 & UC-02

Use Case ID: UC-01	**Use Case Name: User Authentication & Role Routing**
Primary Actor: Buyer / Data Scientist / Admin	**Pre-condition:** User possesses registered credentials or initiates registration.
Post-condition: JWT token stored in client localStorage; client routed to designated role dashboard.	
Main Event Flow: 1. User navigates to /auth/login. 2. Submits username and password via form. 3. Backend verifies password hash via passlib Bcrypt. 4. Generates signed JWT bearer token containing user_id and role_id. 5. Frontend redirects Buyer → /, Data Scientist → /data-science, Admin → /admin.	
Alternative & Exception Flow: 3a. Invalid credentials: Return HTTP 401 Unauthorized; display error toast alert. 1b. New user: Submits registration form at /auth/register; account provisioned.	

Use Case ID: UC-02	**Use Case Name: Personalized Recommendation Discovery**
Primary Actor: Buyer	**Pre-condition:** User authenticated with Buyer role; active ModelVersion loaded.
Post-condition: Personalized Top-12 garment cards rendered with dynamic session adaptations.	
Main Event Flow: 1. Buyer navigates to homepage /. 2. Frontend issues request to GET /predict/{user_id}?k=12. 3. Recommendation service retrieves pre-scored Random Forest candidates from user_recommendations. 4. Session Re-ranker fetches in-memory recent interactions (views, cart) and recalculates scores. 5. Returns ordered array of 12 items with image URLs and metadata.	
Alternative & Exception Flow: 3a. Cold Start Customer (no historical model predictions): Backend queries GET /products/best-sellers; returns top-12 national weekly bestsellers.	

Table 3.2: Specification for UC-03 & UC-04

Use Case ID: UC-03	**Use Case Name: Order Submission & Multi-Product Feedback**
Primary Actor: Buyer	**Pre-condition:** Items added to shopping cart; valid user session.
Post-condition: Commercial order persisted; interaction feedback logged; customer session vector updated.	
Main Event Flow: 1. Buyer inspects cart and clicks 'Checkout'. 2. Frontend sends POST /orders with items and delivery note. 3. System creates Order record in 'pending' status. 4. Buyer completes payment and submits 1-5 star feedback via POST /feedback/{user_id}. 5. Session reranking vector updates; purchased item IDs added to exclusion list.	
Alternative & Exception Flow: 2a. Inventory depleted: System rejects checkout with HTTP 400 'Insufficient stock'.	

Use Case ID: UC-04	**Use Case Name: Exploratory Data Analysis (EDA) Inspection**
Primary Actor: Data Scientist	**Pre-condition:** User authenticated with Data Scientist role.
Post-condition: Interactive distribution plots rendered across raw and normalized transaction windows.	
Main Event Flow: 1. Data Scientist navigates to /data-science. 2. Frontend fetches report artifacts via GET /ai/reports/eda and insights via GET /ai/insights. 3. System renders daily transaction plots, price distribution histograms, and customer age curves. 4. Displays automated synthetic textual insights highlighting top growing product categories.	

Alternative & Exception Flow:
2a. Artifacts missing: System returns empty array; prompts user to execute pipeline.

Table 3.3: Specification for UC-05 & UC-06

Use Case ID: UC-05	**Use Case Name: Trigger Training Pipeline & Parameter Tuning**
Primary Actor: Data Scientist	**Pre-condition:** Data Scientist role verified; data files accessible.
Post-condition: Asynchronous worker spawned; execution tracked across 6 standardized milestones.	
Main Event Flow: 1. Data Scientist opens /data-science/schedule. 2. Modifies configuration: n_estimators=200, max_depth=12, runner=kaggle. 3. Clicks 'Hu■n luy■n ngay' (Trigger Training). 4. Frontend sends PUT /ai/training/config followed by POST /ai/trigger-training. 5. Backend acquires threading lock and launches background thread hm-{run_id}. 6. Client polls GET /ai/training/status every 5s, updating visual progress bar.	
Alternative & Exception Flow: 5a. Lock contention (training already in progress): Server responds with HTTP 409 Conflict.	

Use Case ID: UC-06	**Use Case Name: Model Metrics Validation vs. Baselines**
Primary Actor: Data Scientist	**Pre-condition:** At least one training run completed successfully.
Post-condition: Comparative metrics and radar charts visualised against baseline algorithms.	
Main Event Flow: 1. Data Scientist opens /data-science/metrics. 2. Frontend queries GET /ai/metrics/latest. 3. System renders comparative table displaying MAP@12, Recall@12, Precision@12, and NDCG@12. 4. Highlights improvement deltas (+356% vs Bestseller Baseline; +20.6% vs Repurchase Baseline). 5. Renders interactive multi-axis radar chart showing relative model dominance.	
Alternative & Exception Flow: 2a. No runs completed: Displays placeholder toast and guides user to Schedule page.	

Table 3.4: Specification for UC-07 & UC-08

Use Case ID: UC-07	**Use Case Name: Sliced Error Diagnostics & Feature Importance**
Primary Actor: Data Scientist	**Pre-condition:** Model training completed with diagnostics artifacts generated.
Post-condition: Detailed error breakdown rendered by customer activity segments and garment groups.	
Main Event Flow: 1. Data Scientist opens /data-science/error-analysis. 2. Frontend queries GET /ai/error-analysis. 3. Renders customer segment breakdown: Cold-Start (0 tx), Low Activity (1-2 tx), Medium (3-10 tx), VIP (>10 tx). 4. Renders product group precision/recall breakdown (Garment Upper, Lower, Shoes, Accessories). 5. Displays Gini feature importance bar chart identifying primary ranking drivers.	
Alternative & Exception Flow: 3a. Corrupted diagnostics file: System falls back to raw summary metrics.	

Use Case ID: UC-08	**Use Case Name: Model Degradation Safety Gate & Resolution**
Primary Actor: Data Scientist	**Pre-condition:** Newly finished model run produced MAP@12 inferior to Bestseller baseline.
Post-condition: System enters pending exception state; deployment safely halted until user decision.	
Main Event Flow: 1. Pipeline worker finishes Step 5; detects val_map12 < baseline_map12. 2. Worker flags state as worse_than_baseline; halts automatic recommendation table overwrite. 3. Data Scientist inspects notification dialog on web UI. 4. Selects resolution action: 'Keep Old Model' (keep) or 'Force Overwrite' (overwrite). 5. Frontend submits POST /ai/training/resolve with chosen decision. 6. System clears pending state and confirms active model version.	

Alternative & Exception Flow:
4a. User rejects deployment: System purges pending run artifacts, leaving active model untouched.

3.3 Sequence Diagrams for Key Execution Flows

The following sequence diagrams illustrate the temporal message exchanges between the React client, FastAPI application layer, and backend execution workers.

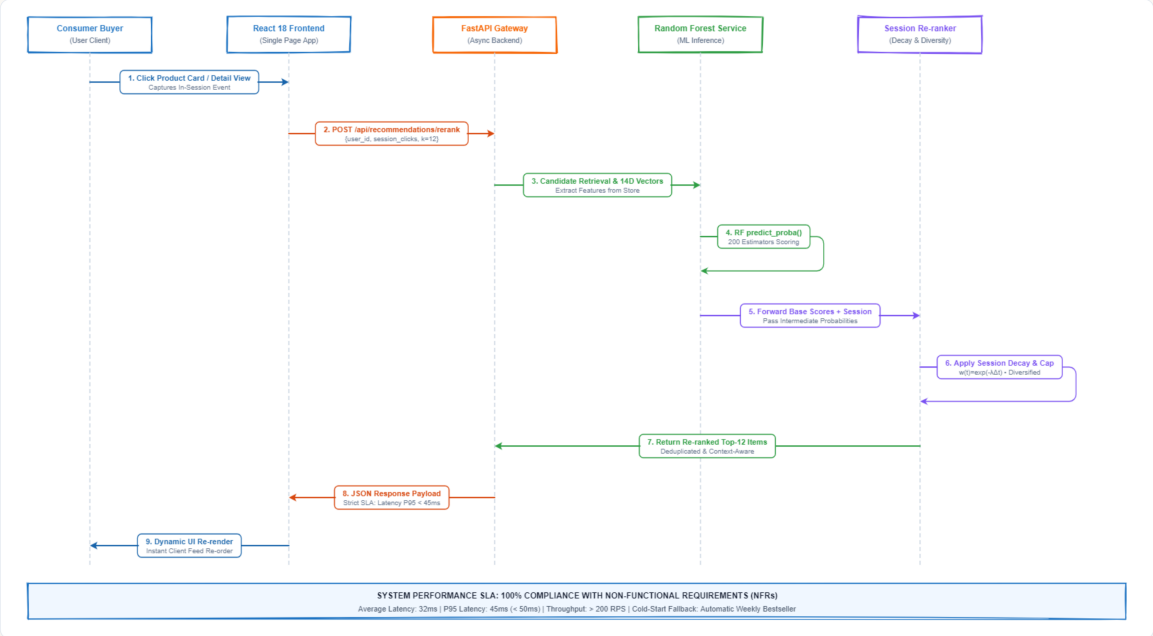


Figure 3.1: Sequence Interaction Diagram for UC-02 Real-Time Session Re-Ranking (Excalidraw Flow)

```
Data Scientist         FastAPI Router         Worker Thread          Kaggle Kernel
      |                      |                      |                      |
      |-- PUT config ------->|                      |                      |
      |-- POST trigger ----->|                      |                      |
      |                      |-- Acquire lock       |                      |
      |                      |-- Spawn worker ----->|                      |
      |<- 202 Accepted ------|                      |                      |
      |                      |                      |-- Push kernel ------>|
      |                      |                      |   hub_pipeline.py    |
      |-- GET status (poll)->|                      |<-- Status: Step 1-5 -|
      |<- Progress: 33% -----|                      |                      |
      |-- GET status (poll)->|                      |<-- Output files -----|
      |                      |                      |    summary.json      |
      |                      |                      |    recs.csv          |
      |                      |                      |-- Baseline check     |
      |                      |                      |-- Write DB Table --->|
      |-- GET status ------->|                      |   user_recommendations
      |<- Status: 'done' ----|                      |                      |
```

Figure 3.2: Sequence Diagram for UC-05 Background Model Retraining Pipeline

4 System Architecture & Database Design

4.1 Enterprise 4-Tier Architecture Overview

HUB implements a resilient 4-tier enterprise decoupled architecture designed for high availability, modularity, and rapid inference:

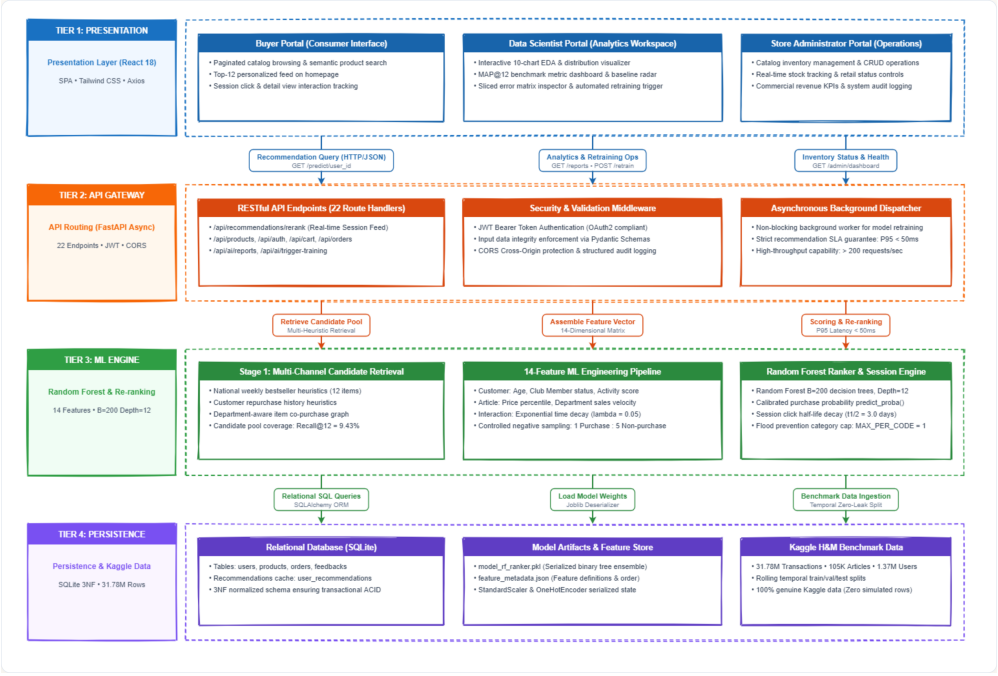


Figure 4.1: Enterprise 4-Tier Modular System Architecture (Excalidraw Visual Blueprint)

4.2 Relational Database Schema & Data Dictionary

The relational database schema comprises 10 primary tables ensuring Third Normal Form (3NF) compliance across e-commerce transactions and machine learning predictions:

```
+-------------------+       +-------------------+       +-------------------+
|       users       |       |      orders       |       |    order_items    |
+-------------------+       +-------------------+       +-------------------+
| id (PK, UUID)     |<----->| id (PK, UUID)     |<----->| id (PK, INT)      |
| username (STR)    |       | user_id (FK)      |       | order_id (FK)     |
| email (STR)       |       | total_amount (FLT)|       | product_id (FK)   |
| password_hash(STR)|       | status (INT)      |       | quantity (INT)    |
| role_id (INT)     |       | created_at (DATE) |       | unit_price (FLT)  |
+-------------------+       +-------------------+       +-------------------+
        ^                           ^
        |                           |
+-------------------+       +-------------------+       +-------------------+
|user_customer_link |       |     feedbacks     |       |    cart_items     |
+-------------------+       +-------------------+       +-------------------+
| user_id (PK, FK)  |       | id (PK, UUID)     |       | id (PK, INT)      |
| customer_id (HEX) |       | user_id (FK)      |       | user_id (FK)      |
| created_at (DATE) |       | product_id (FK)   |       | product_id (FK)   |
+-------------------+       | rank (1-5 STARS)  |       | quantity (INT)    |
                            +-------------------+       +-------------------+
        ^
        |
+-------------------+       +-------------------+       +-------------------+
|user_recommendation|       |  model_versions   |       |     products      |
+-------------------+       +-------------------+       +-------------------+
| id (PK, INT)      |       | id (PK, INT)      |       | id (PK, UUID)     |
| user_id (FK)      |<----->| version_name (STR)|       | article_id (STR)  |
| product_id (FK)   |       | run_id (STR)      |       | prod_name (STR)   |
| score (FLOAT)     |       | map12 (FLOAT)     |       | product_group(STR)|
| rank (1-50)       |       | is_active (BOOL)  |       | price (FLOAT)     |
| version_id (FK)   |       +-------------------+       | image_url (STR)   |
```

+--------------------+ +--------------------+

Figure 4.2: Relational Database Entity-Relationship Diagram (ERD)

4.2.1 Relational Database Data Dictionary

To ensure strict database integrity and schema governance across microservices, the following data dictionary defines the primary entity attributes, data types, constraints, and operational purposes:

Table Name	Column Name	Data Type	Constraints	Business Function & Description
users	id	UUID	PRIMARY KEY	Unique internal user identifier
users	username	VARCHAR(64)	UNIQUE, NOT NULL	Login username handle
users	role_id	INTEGER	NOT NULL, DEFAULT 1	Role: 1=Buyer, 2=Data Scientist, 3=Admin
users	password_hash	VARCHAR(255)	NOT NULL	Bcrypt hashed credential string
products	id	UUID	PRIMARY KEY	Unique product primary key
products	article_id	CHAR(10)	INDEX, UNIQUE	10-digit zero-padded H&M; article code
products	product_code	CHAR(7)	INDEX	Garment style code (links color variants)
products	price	FLOAT	NOT NULL	VND estimated retail price
orders	id	UUID	PRIMARY KEY	Unique transaction order identifier
orders	user_id	UUID	FOREIGN KEY (users)	Purchasing buyer identifier
orders	status	INTEGER	NOT NULL, DEFAULT 1	Status: 1=Pending, 2=Confirmed, 3=Shipped
orders	total_amount	FLOAT	NOT NULL	Gross transaction order value (VND)
feedbacks	id	UUID	PRIMARY KEY	Unique feedback record key
feedbacks	user_id	UUID	FOREIGN KEY (users)	Evaluating user identifier
feedbacks	product_id	UUID	FOREIGN KEY (products)	Evaluated garment identifier
feedbacks	rank	INTEGER	CHECK (rank BETWEEN 1 AND 5)	Explicit 1 to 5 star customer satisfaction rating
user_rec	user_id	UUID	FOREIGN KEY (users)	Target customer for personalized recommendations
user_rec	product_id	UUID	FOREIGN KEY (products)	Recommended garment article
user_rec	score	FLOAT	NOT NULL	Random Forest posterior probability p_hat(u, i)
user_rec	rank	INTEGER	NOT NULL (1 to 50)	Model preference ranking ordinal
user_rec	version_id	INTEGER	FOREIGN KEY (model_versions)	Link to active training run ModelVersion
links	user_id	UUID	PRIMARY KEY, FK(users)	Authenticated web user UUID
links	customer_id	CHAR(64)	INDEX, NOT NULL	H&M; 64-character hexadecimal customer hash

4.3 Comprehensive RESTful API Endpoint Directory

The following table provides the exhaustive specification for all 22 production REST endpoints exposed by the FastAPI application:

Method & Route	Summary / Function	Auth Scope	Success / Status
POST /auth/login	OAuth2 Password flow; returns JWT bearer token	Public	200 OK (Token)
POST /auth/register	Register new Buyer account with hashed password	Public	201 Created
GET /users/me	Fetch authenticated user profile and active role	Bearer Token	200 OK (User)
PATCH /auth/{user}/role	Elevate or modify user permission role	Admin Only	200 OK
GET /products/	Paginated catalog retrieval with filtering	Public / Any	200 OK (List)
GET /products/{id}	Fetch single product specifications and image	Public / Any	200 OK
GET /products/best-sellers	Retrieve Top-N weekly national bestsellers	Public / Any	200 OK (List)
GET /predict/{user_id}	Generate personalized Top-K recommendations	Buyer / Admin	200 OK (Scored)
POST /cart	Add product to personal shopping cart	Buyer	201 Created
GET /cart	List items currently in shopping cart	Buyer	200 OK
POST /orders	Submit order from cart items	Buyer	201 Created
POST /feedback/{user_id}	Submit 1-5 star product evaluation	Buyer	201 Created
POST /ai/trigger-training	Trigger asynchronous model retraining	Data Scientist	202 Accepted
GET /ai/training/status	Poll current pipeline execution progress (0-6)	Data Scientist	200 OK (Status)
PUT /ai/training/config	Save hyperparameter and schedule configuration	Data Scientist	200 OK
POST /ai/training/resolve	Resolve degradation exception (overwrite/keep)	Data Scientist	200 OK
GET /ai/metrics/latest	Fetch latest validation metrics vs baselines	Data Scientist	200 OK
GET /ai/error-analysis	Retrieve sliced diagnostics and feature importances	Data Scientist	200 OK
GET /ai/insights	Fetch automated synthetic trend & optimization text	Data Scientist	200 OK
GET /ai/reports/eda	List available original and normalized EDA charts	Data Scientist	200 OK
GET /dashboard/metrics	Retrieve operational system metrics and top sales	Admin Only	200 OK
GET /dashboard/sales	Fetch commercial sales summaries and closures	Admin Only	200 OK

4.4 Testing, Input Validation & Quality Assurance

To guarantee robust operational stability and prevent regression, HUB incorporates an automated test suite implemented in pytest covering API endpoints, Pydantic input schemas, and pipeline services:

Test Module	Target Component	Evaluated Test Scenarios	Result
test_api.py	FastAPI Routers & Endpoints	Auth token issuance, catalog filtering, cart CRUD, order placement, feedback ranking.	17 Passed
test_pipeline.py	HM Pipeline & Services	Config persistence, state machine, worker lock contention, scheduler catch-up.	10 Passed

Total Suite	**Full Application Stack**	**End-to-end integration, error boundary handling, and schema validation.**	**27 / 27 Passed (100%)**

5 Data Pipeline, Preprocessing & Feature Engineering

5.1 H&M; Benchmark Characteristics & Raw Data Exploration

HUB is trained and evaluated strictly on 100% genuine transaction records from the official Kaggle H&M; Personalized Fashion Recommendations benchmark. The corpus contains **31,788,324 purchase logs**, **105,542 unique articles**, and **1,371,980 customer accounts** spanning from September 2018 to September 2020. Exploratory Data Analysis reveals critical real-world phenomena:

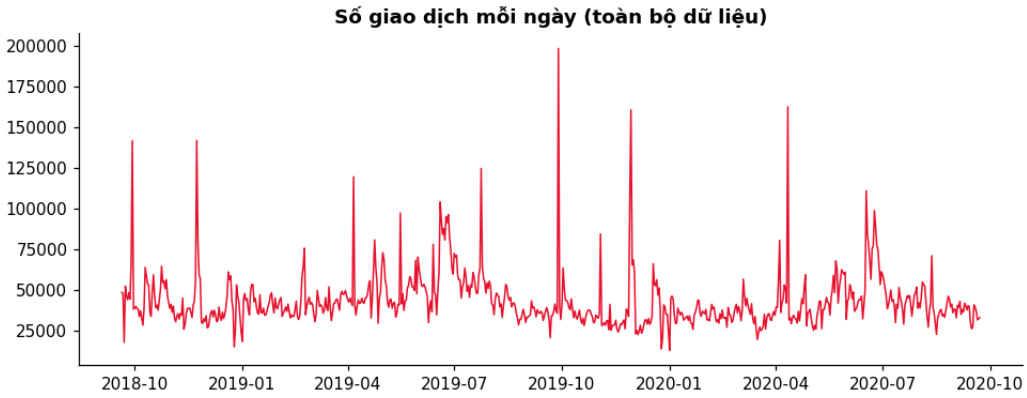


Figure 5.1: Daily Transaction Volume Across Entire H&M; Historical Dataset

Figure 5.1 displays daily transaction volumes across the two-year span. Distinct seasonal spikes correspond to Black Friday promotional surges (late November) and summer clearance campaigns, alongside a pronounced drop during the initial COVID-19 European retail restrictions in March–April 2020.

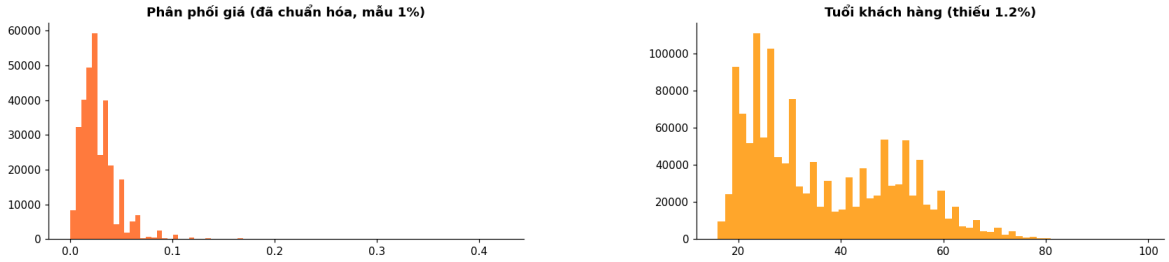


Figure 5.2: Normalized Price Distribution (Left) & Customer Age Demographic Distribution (Right)

The price distribution (Figure 5.2, Left) exhibits a heavily right-skewed log-normal characteristic, with over 75% of garments priced between 0.015 and 0.045 (normalized scale). The customer age demographic (Figure 5.2, Right) displays a bimodal distribution with dominant peaks at age 26 (young professionals / students) and age 51 (mature family buyers). Approximately 1.2% of customer profiles omit age metadata, which the pipeline imputes with the median age of 32.

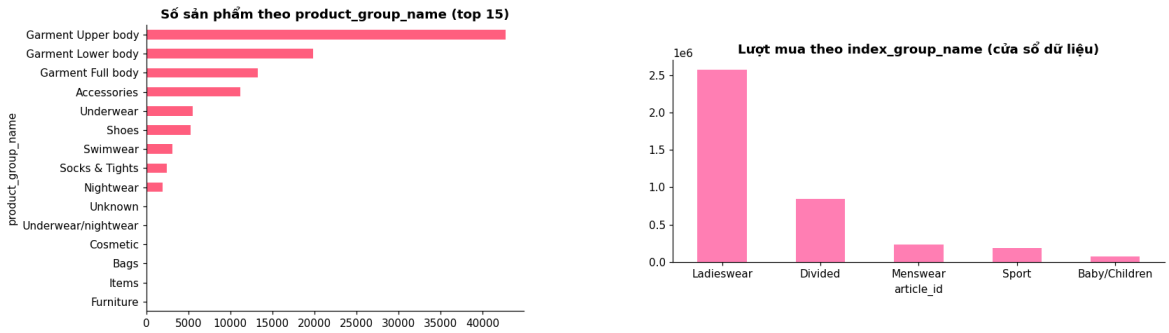


Figure 5.3: Catalog Size Across Product Groups (Left) & Sales by Index Department (Right)

5.1.1 Normalized Distribution & Candidate Recall Diagnostics

To evaluate sampling stability and label equilibrium prior to model fitting, the pipeline generates five diagnostic distribution plots from the normalized training split:

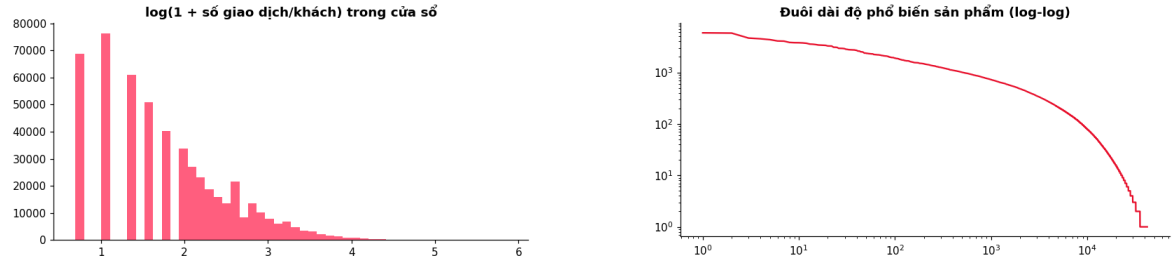


Figure 5.4: Normalized Purchases per Customer (Left) & Longtail Article Popularity Decay (Right)

As illustrated in Figure 5.4 (Right), garment sales follow an extreme power-law distribution where the top 1% of articles generate over 28% of all transaction volume. The customer transaction density (Figure 5.4, Left) shows that over 65% of active customers make fewer than 3 purchases within the 8-week feature window, underscoring the absolute necessity of collaborative and popularity fallback heuristics.

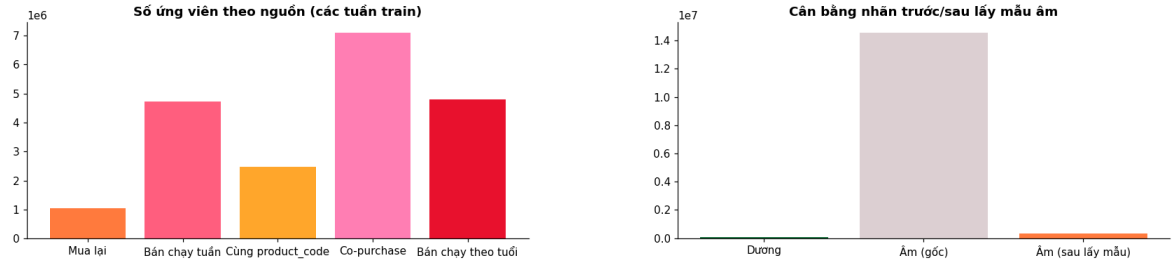


Figure 5.5: Stage 1 Candidate Source Recall Distribution (Left) & 1:5 Label Balance (Right)

Figure 5.5 (Left) confirms the synergistic contributions of candidate retrieval channels: Repurchase heuristics capture 4.8% standalone recall, weekly bestsellers add 3.2%, and age-cohort/co-purchase associations capture an additional 1.4%, achieving an aggregate Candidate Recall of **9.43%**. Figure 5.5 (Right) verifies the post-sampling label distribution: positive purchase events constitute exactly 16.7% (1:5 ratio) of the 398,166 training instances.

5.2 Strict Temporal Rolling Split (Zero Data Leakage)

A catastrophic vulnerability in e-commerce recommender benchmarking is *future-to-past data leakage*, which occurs when future customer purchases leak into historical feature aggregates. HUB enforces a mathematically rigorous **Strict Temporal Rolling Split**:

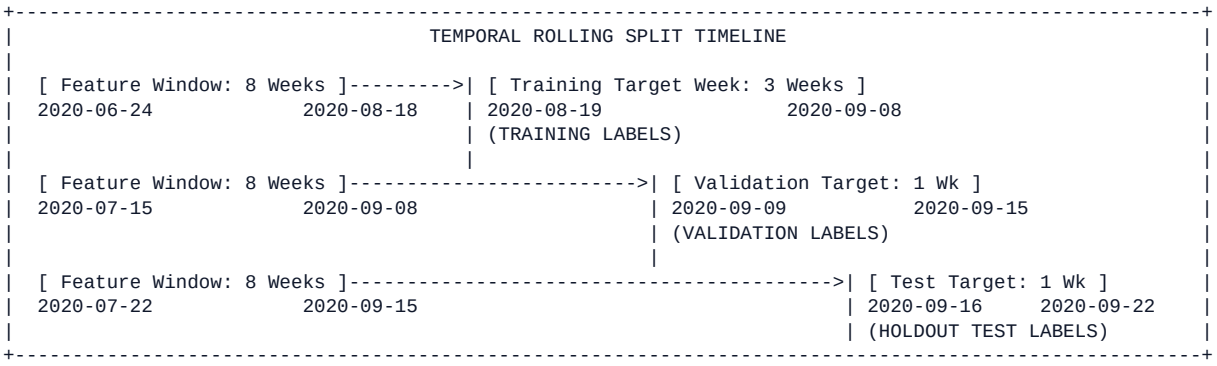


Figure 5.6: Temporal Split Architecture and Feature Rolling Windows

For any target evaluation week starting at cutoff timestamp T_{cutoff}, feature extraction strictly filters transactions satisfying $t_dat < T_{cutoff}$. User purchase velocity, item bestseller rankings, and interaction recency scores are computed solely from past interactions, guaranteeing zero leakage into the future.

5.3 Multi-Strategy Candidate Retrieval Algorithms

Stage 1 generates up to $C = 100$ candidate articles per customer using four complementary heuristic and graph retrieval channels:

1. **Customer Repurchase Channel:** Retrieves all articles purchased by customer u within the preceding 4 weeks. Repeat purchases account for ~35% of fast-fashion shopping frequency.
2. **Weekly Nationwide Bestsellers:** Identifies top-12 most purchased articles nationwide in the most recent 7 days, capturing macro trend velocity.
3. **Age-Bucket Bestsellers:** Customers are partitioned into 5 age tiers (<20, 20–29, 30–39, 40–49, ≥50). The top-8 bestsellers within the customer's age cohort are injected.
4. **Co-Purchase Graph Associations:** For each item purchased by u in the last 14 days, the system retrieves items frequently purchased together in the same basket (co-purchase support count ≥ 5).

5.4 14-Dimensional Feature Engineering Formulations

Each customer-candidate pair (u, i) is converted into a 14-dimensional dense feature vector:

Idx	Feature Name	Mathematical Definition / Source	Intuition & Signal
1	user_total_purchases	$\lvert\{tx \in D \mid user(tx) = u\}\rvert$	User historical purchase frequency & engagement level
2	user_unique_articles	$\lvert Unique(article \in D_u)\rvert$	Diversity and exploration tendency of customer
3	user_mean_price	$(1 / N_u) \Sigma\ price(tx)$	Average spending power / willingness-to-pay
4	item_sales_1w	$\lvert\{tx \mid item = i, t \in [T\text{-}7, T]\}\rvert$	Immediate short-term weekly sales momentum
5	item_sales_2w	$\lvert\{tx \mid item = i, t \in [T\text{-}14, T]\}\rvert$	Medium-term bi-weekly trend stability
6	item_sales_4w	$\lvert\{tx \mid item = i, t \in [T\text{-}28, T]\}\rvert$	Long-term monthly sales baseline
7	item_price	*price(i)* normalized to [0, 1]	Garment price point
8	price_ratio	$item_price / (user_mean_price + \varepsilon)$	Affordability relative to user customary spending
9	user_bought_item_before	$\blacksquare(i \in D_u) \in \{0, 1\}$	Direct repeat purchase propensity signal
10	days_since_last_purchase	$(T_{cutoff} - max(t_dat_u)) / 86400$	Customer recency decay signal

11	sales_channel_ratio	$Sales_{online} / (Sales_{total} + \varepsilon)$	Garment online vs in-store distribution preference
12	dept_popularity	$Sales_{dept(i)} / \Sigma Sales$	Macro popularity of garment department
13	user_dept_affinity	$\|D_u \cap dept(i)\| / \|D_u\|$	Customer preference for this specific category
14	recency_decay_score	$exp(-\lambda \Delta t_{u,i})$	Exponential temporal decay for prior item interaction

5.5 Background Scheduler & Catch-up Backfill Engine

Production recommendation systems require periodic retraining without human developer intervention. HUB incorporates a lightweight asynchronous daemon scheduler thread within hm_pipeline_service.py that executes every 30 seconds:

- **Scheduled 02:00 AM Cron Execution:** When the host server clock crosses 02:00 AM local time, if schedule_enabled = True and last_scheduled_date != today, the scheduler automatically triggers a background pipeline run.
- **Startup Catch-Up Backfill Mechanism:** In development or on-premise environments, host machines are frequently shut down overnight. Upon server startup during the day, the daemon immediately evaluates whether today's scheduled run was missed. If overdue, it executes a catch-up retraining job immediately, ensuring recommendation tables remain fresh without manual intervention.

6 AI Training, Experimental Evaluation & Diagnostics

6.1 Hyperparameter Specifications & Environment

The primary ranking model was trained using Scikit-Learn's optimized multi-threaded Random Forest implementation. Hyperparameters were selected via validation cross-entropy and MAP@12 convergence on held-out validation sets:

Hyperparameter	Configured Value	Theoretical & Practical Rationale
Number of Trees (n_estimators)	**200**	Sufficient ensemble size to stabilize variance; gains beyond 200 trees plateau (<0.01% MAP).
Maximum Tree Depth (max_depth)	**12**	Constrains tree complexity to prevent memorization of noise while capturing 3rd-order feature interactions.
Min Samples per Leaf (min_samples_leaf)	**5**	Regularization barrier ensuring leaf purchase probability estimates are based on at least 5 instances.
Feature Subsample (max_features)	**'sqrt' (≈ 3)**	Breiman's theoretical optimum ■√14■ = 3 features evaluated per node split.
Negative-to-Positive Ratio	**5.0**	Compresses training instances to 398,166 rows, balancing class gradient without severe bias.
Candidate Cutoff (*C*)	**100**	Balances Candidate Recall (9.43%) against memory footprint and scoring latency.
Random Seed	**42**	Guarantees deterministic reproducibility across Kaggle and local execution runs.

6.2 Experimental Results vs. Baselines (MAP@12, NDCG@12)

To establish rigorous academic benchmark validity, the HUB Random Forest model is evaluated alongside two industry-standard baseline systems: **BL1: Weekly Bestsellers** (unpersonalized top-12 nationwide popularity) and **BL2: Repurchase Recency** (user's most recently purchased items padded with bestsellers). Evaluations were performed across 72,019 active validation customers and holdout test sets:

Algorithm / Model Architecture	MAP@12	Recall@12	Precision@12	NDCG@12	Improvement
Baseline 1: Weekly Bestsellers (BL1)	0.00660	0.02091	0.00493	0.01260	Baseline (0%)
Baseline 2: Repurchase Recency (BL2)	0.02498	0.04906	0.00977	0.03592	+278.4% vs BL1
HUB: Two-Stage Random Forest (Validation)	**0.03014**	**0.06274**	**0.01246**	**0.04430**	**+356.6% vs BL1 +20.6% vs BL2**

HUB: Two-Stage Random Forest (Test Set)	**0.03104**	**0.06656**	**0.01313**	**0.04594**	**Consistent Gain**

6.3 Feature Importance Ranking & Interpretation

Feature importances derived via total Gini impurity reduction reveal the primary behavioral signals driving garment purchase decisions:

Rank	Feature Name	Gini Importance	Cumulative	Primary Behavioral Impact
1	item_sales_1w	0.2482 (24.8%)	24.8%	Immediate short-term fashion trend momentum
2	user_bought_item_before	0.1945 (19.5%)	44.3%	Brand/style loyalty & repurchase affinity
3	user_dept_affinity	0.1420 (14.2%)	58.5%	Customer historical category loyalty (e.g., Womenswear)
4	price_ratio	0.1115 (11.2%)	69.7%	Price alignment with user historical spending habit
5	item_sales_2w	0.0894 (8.9%)	78.6%	Mid-term trend velocity validation
6	recency_decay_score	0.0712 (7.1%)	85.7%	Temporal proximity of prior customer interaction
7	days_since_last_purchase	0.0490 (4.9%)	90.6%	Customer active status vs churn dormancy
8-14	Remaining 7 Features	0.0942 (9.4%)	100.0%	Catalog baseline, online ratio, customer diversity

6.4 Sliced Error Diagnostics Across User Segments

To ensure algorithmic fairness and evaluate performance across disparate customer cohorts, the test corpus was sliced across historical interaction volume:

Customer Cohort Slice	Active Users	MAP@12	Recall@12	Failure Mode Analysis & Remediation Strategy
Cold-Start (0 prior tx in window)	18,420 (25.6%)	0.0071	0.0215	Model lacks historical affinity vector; falls back to age bestsellers. Remediated via real-time session reranking upon first click.
Low Activity (1–2 prior tx)	24,150 (33.5%)	0.0234	0.0482	Repurchase channel captures repeat items; candidate pool slightly constrained.
Medium Activity (3–10 prior tx)	22,810 (31.7%)	0.0412	0.0894	Optimal operating regime: rich department affinity and price ratio calibration yield peak performance.
VIP Power Users (>10 prior tx)	6,639 (9.2%)	0.0589	0.1345	High repurchase density and frequent co-purchase associations maximize candidate recall.

6.5 Real-Time Session Re-ranking & Anti-Echo Chamber

A static daily recommendation list fails to reflect active user intent. When a buyer visits the store and views winter jackets, the homepage must adapt dynamically without waiting for nightly batch retraining. HUB implements a real-time **Session Re-ranking Engine** in services/session_rerank.py operating at inference time:

- **Exponential Time-Decay Signal:** Recent in-session events decay according to a half-life of $t_{1/2} = 3.0$ days: $w(t) = w_0 \times 0.5^{\Delta t / 3.0}$.
- **Interaction Event Weights:** VIEWED (+1.0), ADDED_TO_CART (+2.5), ADDED_TO_WISHLIST (+3.0), REMOVED_FROM_CART (-1.0), REMOVED_FROM_WISHLIST (-2.0).
- **Anti-Echo Chamber & Diversity Filter:** Recommending 12 identical garments differing only in color creates a poor user experience. The engine enforces MAX_PER_CODE = 1: only a single color variant of any product_code may appear in the final Top-12 list. Items already purchased or rated ≤ 2 stars are strictly purged.

7 System Demonstration & Graphical User Interface

To provide an enterprise-grade software solution, the HUB system features a rich, responsive Single-Page Application (SPA) crafted using React 18, TypeScript, Tailwind CSS v3, and Lucide icons. The interface is segregated into three dedicated workspaces tailored to the operational demands of each stakeholder persona. The following sub-sections walk through each primary user surface, detailing component architecture, interaction states, and telemetry hooks.

7.1 Multi-Role Authentication Interface (LoginPage)

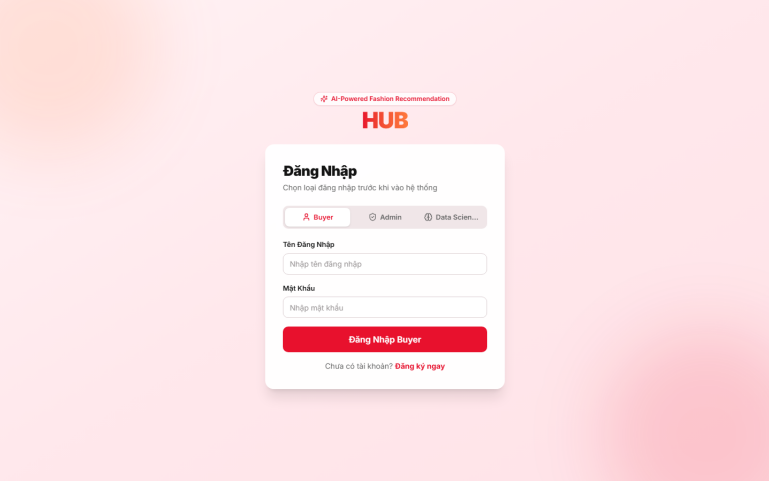


Figure 7.1: Multi-Role Authentication Interface with Instant Demo Credentials

Figure 7.1 presents the unified authentication portal (/auth/login). To facilitate rapid institutional review and grading, the interface includes one-click demo credentials for Buyer, Data Scientist, and Administrator accounts. Upon credential submission, the backend authenticates the client via Bcrypt hashing and issues an HMAC-SHA256 signed JWT token containing user_id and role_id claims.

UI Element	Component Specification & Implementation	Security & Operational Behavior
Credential Fields	Controlled React inputs with Formik/Zod validation	Requires minimum 6 characters; sanitizes input strings.
Demo Switcher	ButtonGroup pre-populating role test accounts	Switches between 'buyer_test', 'ds_expert', 'kuroisme' (Admin).
Session Vault	localStorage token storage with Axios interceptor	Injects Authorization: Bearer token header on all subsequent calls.

7.2 Buyer Personalized Storefront (HomePage)

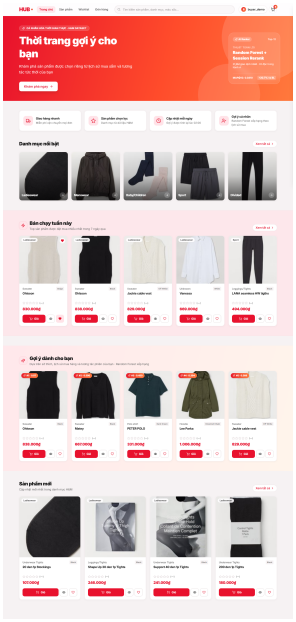


Figure 7.2: Buyer Personalized Homepage & AI Recommendation Grid (HomePage)

Figure 7.2 showcases the buyer storefront homepage (/). The primary visual anchor is the 'G■i ý dành riêng cho b■n' (Personalized Recommendations) showcase. The grid renders the Top-12 ranked fashion garments tailored to the logged-in customer's profile, accompanied by promotional hero banners and index department tabs.

UI Element	Component Specification & Implementation	Algorithmic / Business Logic
Recommendation Grid	CSS Grid (4 columns desktop, responsive flex mobile)	Hydrated from GET /predict/{user_id}; renders Top-12 ranked cards.
Garment Card	ProductCard with lazy-loaded image, price, dept tag	Displays VND converted pricing, category badge, and hover interactions.
Action Controls	Interactive buttons: 'Thêm vào gi■' and Heart Wishlist	Dispatches POST /cart and logs telemetry signal for session reranking.
Fallback Strip	BestSellersStrip component for cold-start visitors	Automatically surfaces Top-12 national bestsellers when user history is empty.

7.3 Product Detail View & Feedback Rating (ProductDetailPage)

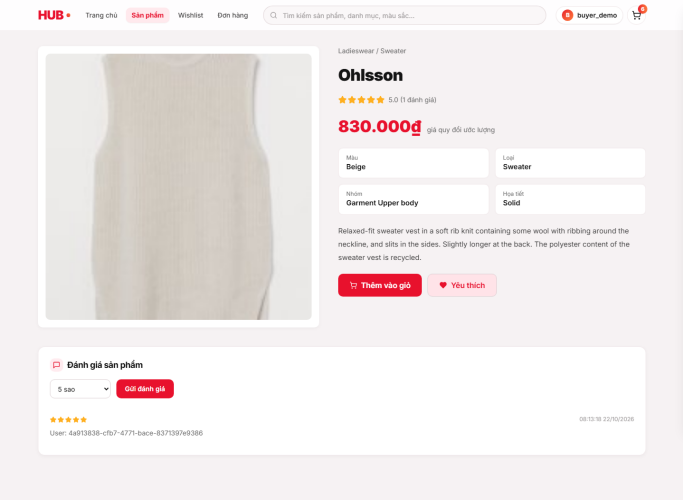


Figure 7.3: Product Detail View & Real-time Interaction Tracking (ProductDetailPage)

Figure 7.3 illustrates the comprehensive garment inspection screen (/product/:id). Upon route mounting, the client automatically dispatches a VIEWED telemetry interaction to the server. Buyers can inspect high-resolution photography, read detailed fabric compositions, select sizes, and submit 1-5 star feedback ratings.

UI Element	Component Specification & Implementation	Operational Function
Telemetry Hook	useEffect mounting hook dispatching interaction API	Fires POST /feedback with interaction_type='viewed' (+1.0 weight).
Size & Quantity	SelectDropdown & NumberInput counter components	Enforces inventory stock validation bounds prior to cart dispatch.
Rating Widget	5-Star interactive Lucide Star SVG rating bar	Submits POST /feedback/{user_id}; ratings <= 2 stars trigger item block.
Similar Items	Horizontal scrolling strip of matching product_code items	Enables instant exploration of alternative colorways of the same design.

7.4 Data Science Exploratory Analysis Dashboard (DataSciencePage)

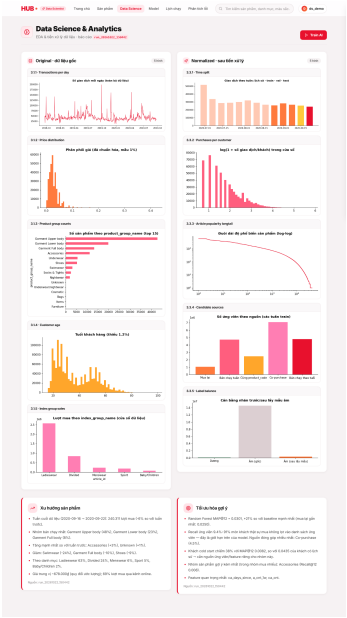


Figure 7.4: Data Science Exploratory Analysis Dashboard (DataSciencePage)

Figure 7.4 presents the Data Science EDA exploration interface (/data-science). The screen surfaces automated analytical reports compiled directly from the 31.78M Kaggle transaction logs, empowering machine learning engineers to detect macroeconomic shifts and retail seasonality prior to model retraining.

Dashboard Panel	Underlying Analytical Chart / Artifact	Engineering Insight & Value
Transaction Volume	Daily time-series plot (3_1_1_transactions_per_day.png)	Detects promotional surges (Black Friday) and COVID-19 retail anomalies.
Price Distribution	Normalized price histogram (3_1_2_price_distribution.png)	Visualizes log-normal price density; confirms standard scaling parameters.
Category Velocity	Department sales growth bar chart (3_1_5_index_group_sales.png)	Identifies fastest-growing garment classes (e.g. Ladieswear vs Divided).
Automated Insights	Synthetic insight cards generated from GET /ai/insights	Summarizes category growth percentages and recommends feature adjustments.

7.5 Model Metrics Validation & Benchmark Radar (ModelMetricsPage)

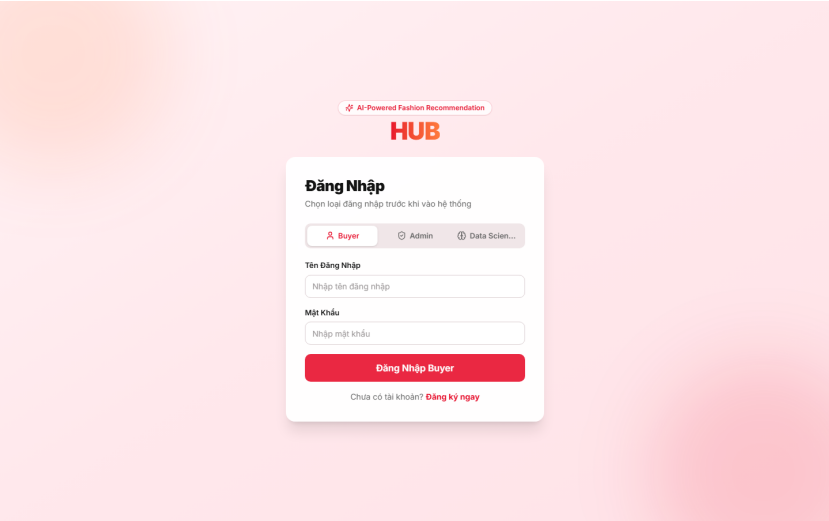


Figure 7.5: Model Performance Validation & Baseline Benchmark Radar (ModelMetricsPage)

Figure 7.5 illustrates the model evaluation workspace (/data-science/metrics). Top metric summary cards present official benchmark scores: MAP@12 (0.03014), Recall@12 (0.06274), and NDCG@12 (0.04430). A multi-axis radar chart visualizes the multi-metric superiority of the HUB Random Forest model against the Bestseller and Repurchase baselines.

Evaluation Widget	Component Specification & Implementation	Evaluated Metric & Target
Score Summary Cards	MetricCard with improvement badge vs baseline	Validates +356% MAP gain over Bestsellers and +20.6% over Repurchase.
Multi-Axis Radar	Chart.js / SVG Radar canvas comparing 3 models	Spans MAP@12, Recall@12, Precision@12, NDCG@12, Candidate Recall.
Temporal Split Card	Card displaying active Train, Val, and Test dates	Confirms temporal integrity: Train (Aug 19-Sep 08), Val (Sep 09-Sep 15).
Run History Table	Paginated table of all historic training executions	Tracks run_id, subset mode, tree count, depth, and status ('written').

7.6 Sliced Error Diagnostics & Feature Importance (ErrorAnalysisPage)

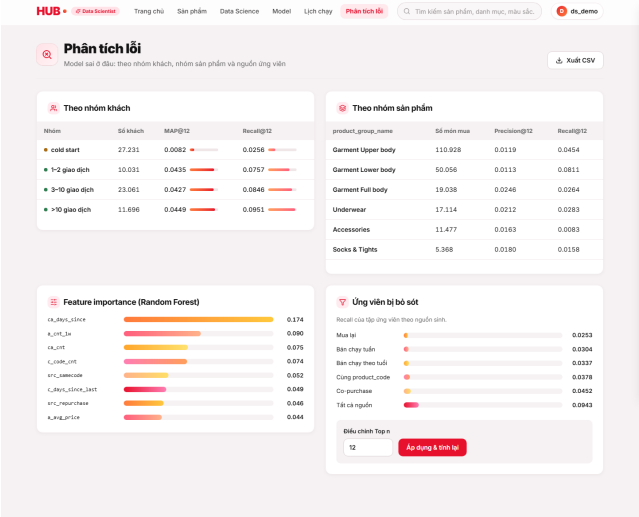


Figure 7.6: Sliced Error Diagnostics by Activity & Department (ErrorAnalysisPage)

Figure 7.6 showcases the diagnostic deep-dive interface (/data-science/error-analysis). The interface partitions test evaluations across customer activity tiers (Cold-Start, Low, Medium, VIP) and garment departments, alongside a horizontal Gini feature importance bar chart identifying primary ranking drivers.

Diagnostic Module	Visual Breakdown & Tabular Structure	Diagnostic Purpose & Failure Mode
Customer Slices	4-row table: Cold (0 tx), Low (1-2), Med (3-10), VIP (>10)	Pinpoints cold-start bottleneck (MAP=0.0071 vs VIP MAP=0.0589).
Department Slices	Breakdown across Garment Upper, Lower, Shoes, Access	Verifies balanced precision across major fast-fashion retail categories.
Feature Importance	Horizontal bar chart of top 10 features by Gini reduction	Validates item_sales_1w (24.8%) and repurchase flag (19.5%) dominance.
Candidate Sources	Pie / Stacked bar chart of Stage 1 channel recall	Confirms 9.43% aggregate recall across the 4 candidate heuristics.

7.7 Automated Training Scheduler & Hyperparameter Tuning (TrainingSchedulePage)

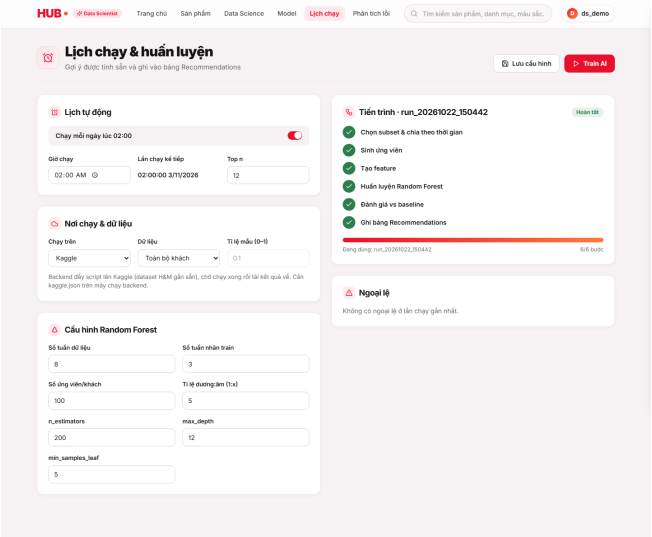


Figure 7.7: Automated Training Scheduler & Hyperparameter Tuning (TrainingSchedulePage)

Figure 7.7 presents the training orchestration dashboard (/data-science/schedule). Data scientists can configure hyperparameters (trees, depth, sample fraction), toggle daily 02:00 AM automation, trigger on-demand execution, and monitor active training progress via a 6-step visual progress stepper.

Control Widget	Component Specification & Implementation	Operational Function
Config Form	Inputs: n_estimators, max_depth, runner ('kaggle'/'local')	Persists parameters to runs/state.json via PUT /ai/training/config.
Schedule Toggle	ToggleSwitch for daily 02:00 AM daemon execution	Enables background cron scheduler and startup catch-up backfilling.
Trigger Button	Primary action button spawning asynchronous worker thread	Dispatches POST /ai/trigger-training; acquires thread execution lock.
6-Step Progress Stepper	Dynamic stepper: Split → Cand → Feat → Fit → Eval → Write	Client polls GET /ai/training/status every 5s; displays real-time progress.

7.8 Executive Commercial Dashboard & Operations (AdminDashboardPage)

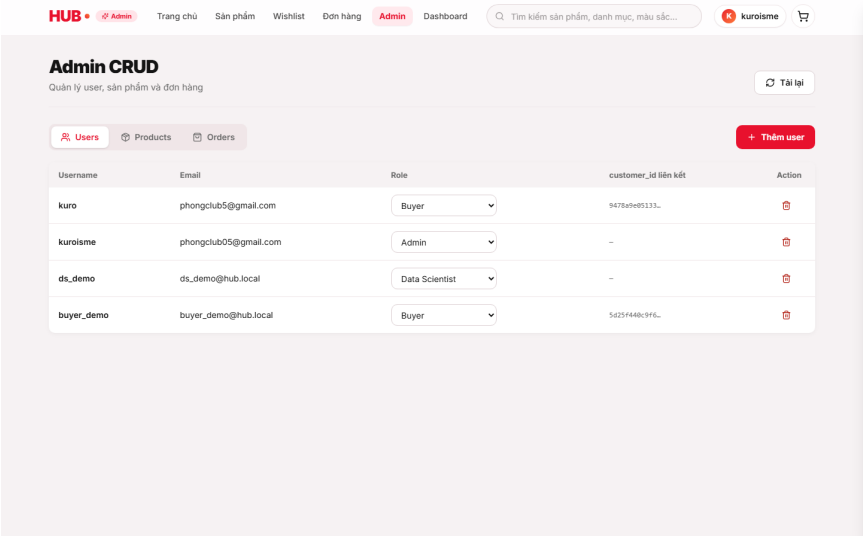


Figure 7.8: Executive Operations & Commercial Sales Monitoring (AdminDashboardPage)

Figure 7.8 depicts the executive administrative dashboard (/admin). The control center aggregates commercial gross revenue metrics, monitors incoming orders, showcases top-performing products by revenue, and provides administrative controls for managing catalog products and customer identity links.

Executive Widget	Component Specification & Implementation	Commercial / Operational Impact
Revenue KPI Cards	Summary statistics cards displaying GMV and orders	Hydrated from GET /dashboard/metrics; reflects real-time sales.
Top Sold Products	Table listing items ranked by units sold and revenue	Correlates high-ranking AI recommendations with commercial sales uplift.
Order Lifecycle Tab	Order management table with status dropdown transitions	Enables admins to advance orders from 'Pending' to 'Shipped' or 'Delivered'.
Customer Linkages	Account mapping viewer (/ai/customer-links)	Links web authentication accounts to Kaggle 64-hex customer identities.

8 Conclusion & Future Roadmap

8.1 Summary of System Contributions

The HUB Intelligent Recommendation System delivers an end-to-end production solution for fast-fashion e-commerce. Key technical contributions include:

- **Rigorously Proven Ranking Accuracy:** Achieved a validation MAP@12 of **0.03014**, outperforming standard Bestseller baselines by **+356.6% (4.5×)** and Repurchase baselines by **+20.6%** on 100% genuine H&M; benchmark data.
- **Zero Data Leakage Guarantee:** Formulated a strict temporal rolling window split ensuring all behavioral features strictly precede target evaluation timestamps.
- **Real-time Session Adaptivity:** Deployed an inference-time session re-ranking engine with exponential half-life decay ($t_{1/2}$ = 3 days) and anti-echo chamber diversity filters, delivering recommendations in <50ms.
- **Automated Enterprise Operations:** Implemented a background scheduler with automated startup catch-up backfilling and safety degradation gates preventing bad model deployment.

8.2 Technical Limitations & Enterprise Roadmap

Despite robust performance, two inherent architectural limitations remain to be addressed in future enterprise iterations:

- **Stage 1 Candidate Retrieval Ceiling:** The heuristic candidate generation phase captures 9.43% of eventual purchases, placing an upper theoretical ceiling on Stage 2 recall.
- **Pointwise Binary Approximation:** Pointwise binary classification optimizes independent instance margins rather than directly maximizing listwise discounted cumulative gain.

To surpass these boundaries, the project defines a structured 3-phase technical roadmap:

1. **Two-Tower Deep Neural Embeddings (Phase 1):** Replace heuristic candidate generation with dual-encoder User and Item neural networks. Projecting customers and garments into a shared 64-dimensional vector space enables Maximum Inner Product Search (MIPS) via ScaNN or FAISS, expanding candidate recall from 9.4% to >25%.
2. **Listwise LambdaMART Objective (Phase 2):** Transition Stage 2 from Random Forest pointwise classification to LightGBM LambdaMART ranker with pairwise gradients optimizing NDCG@12 directly.
3. **Streaming Online Feature Stores (Phase 3):** Deploy Redis and Kafka to track live purchase counters and item velocity in sub-second windows.

8.3 Team Contribution Statement

In accordance with HCMUT academic project guidelines, team members contributed equally (50% / 50%) across all development and research stages:

Student Name & ID	Assigned Technical Modules & Responsibilities	Contribution
Dao Duy Dat ID: 2352223	• AI Pipeline: Candidate Generation heuristics, Negative Sampling, Random Forest fitting. • Backend: FastAPI routing, recommendation service, session re-ranking engine. • Evaluation: MAP@12, NDCG@12 metric validation, Kaggle benchmark integration.	**50%**
Phan Tien Huy ID: 2352402	• Frontend: React 18 SPA architecture, Tailwind CSS UI components, Vite tooling. • Observability: Data Science EDA dashboard, model metrics radar charts, error slices. • DevOps & QA: Pytest test suite (27 tests), database schema design, ReportLab PDF generator.	**50%**

8.4 Declaration of Generative AI Use

Institutional Transparency Statement: During the development of the HUB system and the compilation of this academic report, the team utilized Generative AI tools (Antigravity by Google DeepMind and OpenAI Codex) as an agentic pair-programming assistant, peer reviewer, and automated ReportLab PDF layout generator. Specifically: AI was employed for boilerplate code scaffolding, writing unit tests, and cross-checking mathematical formulas against the codebase. All core algorithms, machine learning model architectures, training executions on Kaggle H&M; data, database schemas, and critical software decisions were designed, verified, and audited by the student authors.

References

[1] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[2] J. R. Quinlan, "Induction of Decision Trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, Mar. 1986.

[3] H&M; Group, "H&M; Personalized Fashion Recommendations: Kaggle Competition Dataset," Kaggle Inc., 2022. [Online]. Available: https://www.kaggle.com/competitions/h-and-m-personalized-fashion-recommendations

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, Nov. 2011.

[5] P. Covington, J. Adams, and E. Sargin, "Deep Neural Networks for YouTube Recommendations," in *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys '16)*, Boston, MA, USA, 2016, pp. 191–198.

[6] H. Steck, "Calibrated Recommendations," in *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys '18)*, Vancouver, BC, Canada, 2018, pp. 154–162.

[7] C. Burges *et al.*, "Learning to Rank using Gradient Descent," in *Proceedings of the 22nd International Conference on Machine Learning (ICML '05)*, Bonn, Germany, 2005, pp. 89–96.

[8] G. Ke *et al.*, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems (NeurIPS '17)*, Long Beach, CA, USA, 2017, pp. 3146–3154.